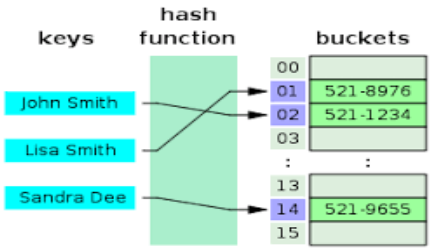


Parallel hashtable





Last update: 14 Mai 2023
Publicare: 14 Mai 2023
Soft deadline: **28 Mai 2023**
Hard deadline: **2 Iunie 2023**

Se va implementa o structură de date tip hashtable folosind CUDA.

Această structură de date tip hashtable va avea următoarele caracteristici:

- va face maparea key -> value, (o singură valoare per key)
- va putea stoca date de tip int32, mai mari strict ca 0, atât pentru chei, cât și pentru valori
  - valid (key->value): 1234 -> 8965, 12334244 -> 8223965
  - invalid (key->value): 0 -> 9779, -12334244 -> 83965, 7868 -> -977, “jknjjk” -> 78
- va stoca datele în VRAM și va putea să își reajusteze dimensiunea cât să poată acomoda numărul de perechi key->value, având un loadFactor decent (sloturi ocupate / sloturi disponibile)
- va rezolva intern coliziunile (exemplu: prin resize, folosind multe funcții de hash etc.)
  - funcțiile hash vor fi pentru cazurile generale (ex. cele de tip ax % b, a si b prime)
  - exemplu de funcție hash neacceptată: h(x) = x % limit, atunci când cheile sunt tot timpul în ordine crescătoare pe intervale și unice (nu tratează cazul general)
- va face update la valoarea unei chei -> operația de insert nu va conține niciodată 2 chei identice, însă operații succesive de insert nu au restricții (se va face update de valoare)
- va putea întoarce corect, într-un timp rapid, valoarea corespunzătoare unei chei
- va întoarce încărcarea (memoria efectiv folosită / memoria alocată) via funcția load_factor
  - o implementare greșită sau neclară poate duce la o depunctare suplimentară

Implementarea temei se va face folosind scheletul pus la dispoziție.
Alocările de memorie interne pentru hashtable vor fi făcute exclusiv prin funcțiile wrapper din schelet, anume `_cudaMalloc, _cudaMallocManaged, _cudaFree.`

Exemple de implementări:

- **Linear probing** - se calculează hash(key) și se dacă slotul nu e liber se încearcă succesiv hash(key) + 1… hash(key) + N
- **Cuckoo based** - se folosesc 2-3 funcții hash și se caută un slot liber, alternativ se face evict la key2-value2 din acel slot, se inserează nouă pereche key-value, iar key2-value2 se caută să se insereze folosind o altă funcție hash din cele disponibile
- **Bucketized cuckoo** - se folosește tehnica cuckoo, având buckets cu 2, 4, 8 sloturi fiecare.



Schelet tema

Se va completa scheletul pus la dispoziție având următoarele funcții ce trebuie implementate:

```
/* INIT HASH
 */
GpuHashTable::GpuHashTable(int size) { }

/* DESTROY HASH
 */
GpuHashTable::~GpuHashTable() { }

/* RESHAPE HASH
 */
void GpuHashTable::reshape(int numBucketsReshape) { }

/* INSERT BATCH
 */
bool GpuHashTable::insertBatch(int *keys, int* values, int numKeys) { return false; }

/* GET BATCH
 */
int* GpuHashTable::getBatch(int* keys, int numKeys) { return NULL; }

/* GET LOAD FACTOR
 * num elements / hash total slots elements
 */
float GpuHashTable::loadFactor() {
	return 0.f; // no larger than 1.0f = 100%
}

#define HASH_INIT GpuHashTable GpuHashTable(1);
#define HASH_RESERVE(size) GpuHashTable.reshape(size);

#define HASH_BATCH_INSERT(keys, values, numKeys) GpuHashTable.insertBatch(keys,
values, numKeys)
#define HASH_BATCH_GET(keys, numKeys) GpuHashTable.getBatch(keys, numKeys)

#define HASH_LOAD_FACTOR GpuHashTable.loadFactor()
```



Notare

Arhiva ce cuprinde solutia va cuprinde doar fisierele **gpu_hashtable.cu** si **gpu_hashtable.hpp** cat si README.md.

Punctajul maxim este de 100 pct distribuite astfel:

20 pct	Implementare descrisă în README, alături de rezultate și o discuție asupra lor Programul compilează, codul nu are disfuncționalități majore
80 pct	Punctaj dat de bench.py / vmchecker

Arhiva va cuprinde obligatoriu:

- (10 pct aspect cod/solutie) Fișierele cu codul sursa și alte fișiere adiționale folosite, dar minim:
    - gpu_hashtable.cu
    - gpu_hashtable.hpp
- (10 pct explicatie solutie) README.md, unde se explică:
    - Cum s-a implementat soluția ?
    - Cum se stochează hashtable în memoria GPU VRAM ?
    - Output la performanțele obținute și discutie rezultate.